EDAP30 - Lab Report 3

Axel Langenskiöld
`ax2531la-s`

May 25, 2026

1 Part 1: Transfer Learning for Flower Classification

1.1 Methods

Two pretrained image classifiers from `torchvision` were fine-tuned for part1 of the lab. For each backbone the final classification layer was replaced with a 5-way linear head and all other parameters were updated during training.

Experiments:

- *Baseline:* ConvNeXt-Tiny, no augmentation, trained for 10 epochs.
- *Augmentation:* ConvNeXt-Tiny with random resized crop, horizontal flip, color jitter and $\pm 15°$ rotation, trained for 20 epochs.
- *Alternative architecture:* EfficientNet-B0 with the same augmentation pipeline ConvNeXT-Tiny, trained for 20 epochs.

Training

AdamW with lr $= 3 \times 10^{-4}$ and weight decay 10^{-4}, cosine annealing learning-rate schedule, cross-entropy loss, batch size of 32. The checkpoint with the highest validation macro F1 was kept for test evaluation. Training was performed on Apple MPS.

1.2 Results

All three models clear the 90% pass threshold on the test set (Table 1). The augmented ConvNeXt-Tiny achieves the best test macro F1 (0.9588), with the no-augmentation baseline within rounding error (0.9586). EfficientNet-B0 trails by roughly one percent.

Model	Augmentation	Epochs	Val F1	Test F1
ConvNeXt-Tiny (baseline)	no	10	0.9659	0.9586
ConvNeXt-Tiny + aug	yes	20	0.9648	**0.9588**
EfficientNet-B0 + aug	yes	20	0.9636	0.9506

Table 1: Macro F1 on validation and test for each transfer-learning experiment. All three exceed the 90% pass criterion.

1.3 Discussion

The combination of a modern backbone ConvNeXt-Tiny with a fine-tuning pipeline (AdamW, weight decay 10^{-4}, lr $= 3 \times 10^{-4}$, cosine schedule) was sufficient to reach >95% test macro F1 without augmentation. An earlier configuration using ResNet50 with lr $= 10^{-3}$ and no scheduler plateaued around 86% test F1 due to rapid overfitting. The fix that mattered was lowering the learning rate and adding weight decay, far more than swapping in augmentation. Augmentation gave a negligible improvement on top of the already-strong baseline. The likely explanation is that with a stronger backbone, lower learning rate, weight decay and a scheduler, the baseline already generalizes well.

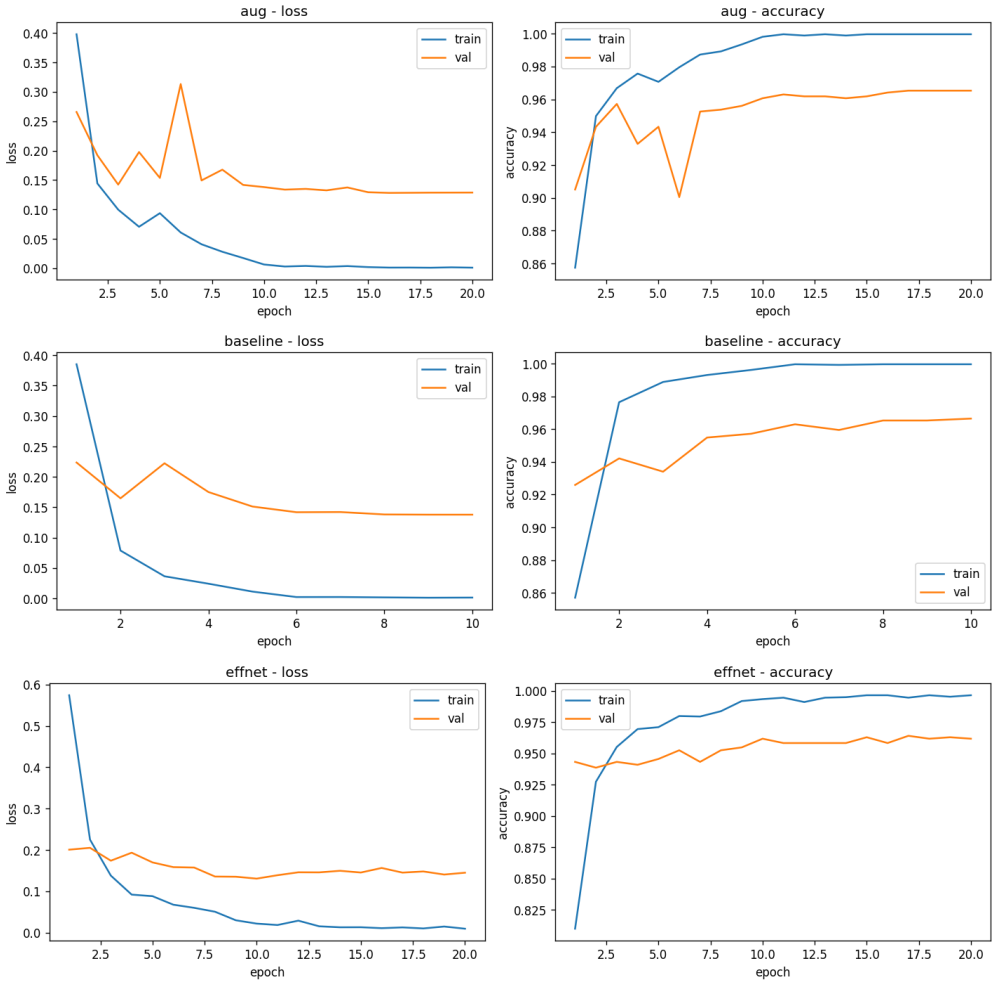

Figure 1: Training and validation loss and accuracy for the three transfer-learning models. Top: baseline ConvNeXt-Tiny (no augmentation, 10 epochs). Middle: ConvNeXt-Tiny with augmentation (best test F1, 20 epochs). Bottom: EfficientNet-B0 with augmentation (20 epochs).

Class-level patterns. The two worst classes were rose (F1 = 0.9453) and tulip (F1 = 0.9421), with most of the confusion between them, which isn't surprising considering they share similar colors and shape. Sunflower was the easiest (F1 = 0.9797), benefiting from it's distinctive yellow color and disc shape.

Weaknesses and ways to improve.

- Tulip and rose remain the bottleneck and a targeted hard-example mining or more aggressive color jitter might separate them better.
- A larger ConvNeXt variant (Small/Base) would likely push test F1 higher, but would come at the cost of longer training time.

1.4 Conclusions

A single, simple fine-tuning recipe applied to three different ImageNet pretrained backbones cleared the 90% pass criterion on the flowers dataset, with the best model (ConvNeXt-Tiny + augmentation) reaching 0.9588 macro F1 on the test set. The dominant lever for hitting this threshold was the training recipe (AdamW, lower learning rate, cosine schedule) , rather than augmentation. Backbone choice still mattered a lot, ConvNeXt-Tiny (2022) consistently outperformed EfficientNet-B0 (2019) on the same data and recipe, supporting the hypothesis that stronger ImageNet pretraining transfers

Class	Precision	Recall	F1	Support
daisy	0.9796	0.9412	0.9600	153
dandelion	0.9624	0.9716	0.9670	211
rose	0.9545	0.9363	0.9453	157
sunflower	0.9667	0.9932	**0.9797**	146
tulip	0.9350	0.9492	0.9421	197
macro avg	0.9596	0.9583	0.9588	864

Table 2: Per-class test metrics for the best model (ConvNeXt-Tiny + augmentation). Sunflower is the easiest class; rose and tulip are the hardest.

to better downstream features. This was especially true when initially using ResNet50 for the baseline, where only a 86-87% was achievable with the training recipe.

2 Part 2: Generative Modelling with LSGAN

2.1 Methods

A convolutional LSGAN was implemented from scratch in PyTorch. The generator maps a 100-dimensional latent vector to a 32×32 grayscale image through four ConvTranspose2d blocks, with BatchNorm and ReLU in the hidden layers and a Tanh output that matches the $[-1, 1]$ normalization used on the real data. The discriminator mirrors this with four strided `Conv2d` layers, LeakyReLU, BatchNorm in the middle layers (no BN on the input layer per DCGAN convention), and a single linear scalar output.

Experiments:

- *FashionMNIST:* 60k 28×28 clothing images, padded to 32×32 with a 2-pixel black border (`transforms.Pad(2)`) and normalized to $[-1, 1]$. Trained for 30 epochs.
- *MNIST:* same pipeline but the dataset is swapped, now using drawn images of numbers.

Training

Adam with lr $= 2 \times 10^{-4}$ and $\beta_1 = 0.5$, $\beta_2 = 0.999$ for both networks and the DCGAN-paper recipe. Batch size 128, latent dimension 100, `NGF=NDF=64`. Weights initialized $\mathcal{N}(0, 0.02)$ per the DCGAN paper. A fixed z is used to render a 64-sample grid every epoch, and both generator and discriminator weights are checkpointed every 5 epochs. Final FID is computed on a full epoch.

2.2 Results

Both runs trained to completion without divergence or collapse. Final FID values clear the ≤ 40 pass threshold by a wide margin (Table 3). MNIST achieves lowest FID for unconditional generation and FashionMNIST sits at a healthy number.

Dataset	Epochs	Train samples	Final FID	Threshold
FashionMNIST	30	59,904	**12.99**	≤ 40
MNIST	30	59,904	**4.67**	≤ 40

Table 3: Final FID on a full training epoch for both LSGAN runs. Both clear the pass threshold; the gap between them reflects MNIST's narrower distribution (10 single-glyph shapes vs. FashionMNIST's varied clothing items).

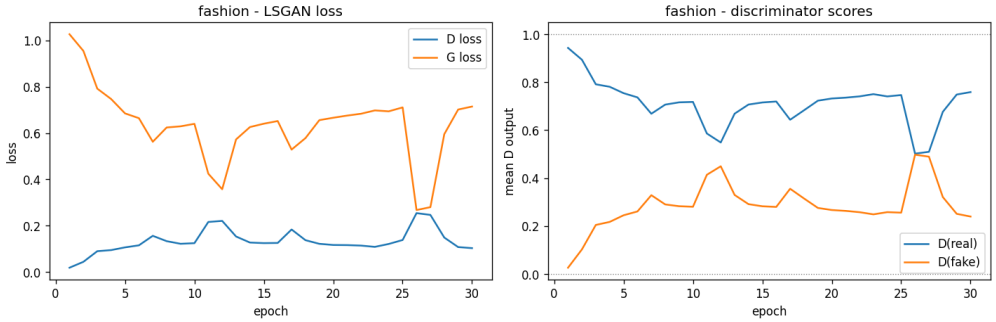

Figure 2: FashionMNIST LSGAN training: generator/discriminator loss (left) and mean discriminator output on real vs. fake batches (right).

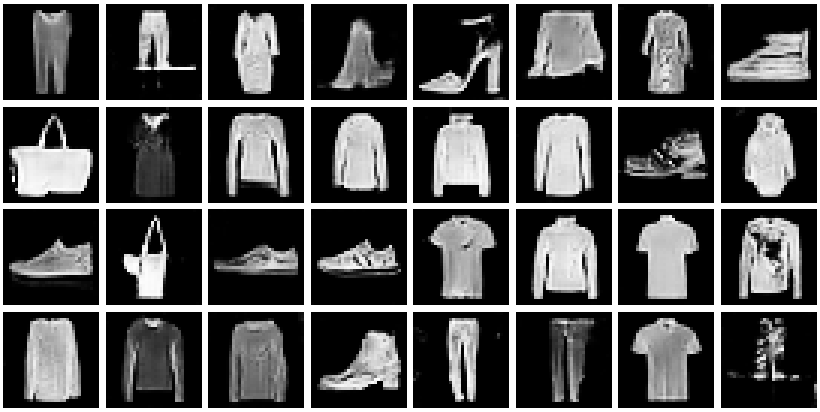

Figure 3: Final 32-image sample grid from the FashionMNIST generator at epoch 30. Multiple clothing categories are recognizable and no visible mode collapse.

2.3 Discussion

LSGAN's MSE loss kept training stable across both datasets. The same pipeline transferred from FashionMNIST to MNIST without any tuning and produced a much lower FID, suggesting the architecture is not overfitting to dataset-specific artifacts and that the choice of training generalizes well.

MNIST's FID is roughly a third of FashionMNIST's (4.67 vs. 12.99), which matches the difficulty gap between the two: MNIST has only 10 single-glyph shapes with limited intra-class variation, while FashionMNIST mixes garments with very different silhouettes and textures that a small DCGAN-scale architecture struggles to render.

2.4 Conclusions

A standard DCGAN architecture trained with the LSGAN adversarial loss was sufficient to clear the FID < 40 pass criterion on both FashionMNIST and MNIST. The fact that the same pipeline worked zero-effort on a second dataset suggests the architecture/recipe combination is robust within low-resolution grayscale image generation. Main takeaway is that GAN training is not so much about tuning and much more about following the DCGAN conventions.

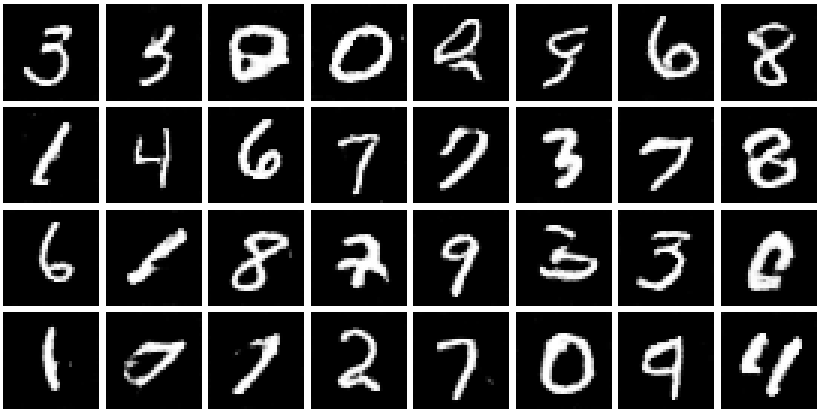

Figure 4: Final 32-image sample grid from the MNIST generator at epoch 30. Digits are sharp and varied, consistent with the very low final FID.